Ejercicio 1

```
def lee_pedidos(ruta_fichero: str) -> list[Pedido]:
    res = []
    with open(ruta_fichero, 'rt', encoding='utf-8') as f:
        lector = csv.reader(f, delimiter=';')
        next(lector)  # Saltar cabecera
        for id_pedido, fecha_hora, mesa, metodo_pago, propina, lineas_str in lector:
            # Transformación de datos
            id_pedido = int(id_pedido.strip())
            fecha_hora = datetime.strptime(fecha_hora.strip(), "%Y-%m-%d %H:%M")
            propina = float(propina.strip())
            # Llamada a la función auxiliar para las líneas de pedido
            lineas = parsea_lista_lineas_pedido(lineas_str)
            res.append(Pedido(id_pedido, fecha_hora, mesa.strip(), metodo_pago.strip(),
                              propina, lineas))
    return res

def parsea_linea_pedido(linea_pedido_str: str) -> LineaPedido:
    """Parsea una línea de pedido en formato 'producto-categoria-cantidad-precio_unitario'"""
    producto, categoria, cantidad, precio_unitario = linea_pedido_str.split("-")
    return LineaPedido(producto.strip(), categoria.strip(), int(cantidad),
float(precio_unitario))

def parsea_lista_lineas_pedido(lineas_pedido_str: str) -> list[LineaPedido]:
    """Parsea la cadena completa de líneas de pedido separadas por '*'"""
    return [parsea_linea_pedido(trozo) for trozo in lineas_pedido_str.split("*")]
```

Test

```
if __name__ == "__main__":
    pedidos = lee_pedidos("./data/pedidos.csv")
    print("\n*********Ejercicio 1*********")
    test_lee_pedidos(pedidos)

    print("\n*********Ejercicio 2*********")
    test_ingreso_total_por_franja(pedidos, 8, 12)
    test_ingreso_total_por_franja(pedidos, 12, 18, "Tarjeta")
    test_ingreso_total_por_franja(pedidos, 18, 24, "Bizum")

    print("\n*********Ejercicio 3*********")
    test_analisis_propinas_ubicacion(pedidos, "Efectivo")
    test_analisis_propinas_ubicacion(pedidos, "Tarjeta")
    test_analisis_propinas_ubicacion(pedidos, "Bizum")

    print("\n*********Ejercicio 4*********")
    test_producto_mas_asociado(pedidos, "Hamburguesa_Completa")
    test_producto_mas_asociado(pedidos, "Cafe_Con_Leche")

    print("\n*********Ejercicio 5*********")
    test_mesas_gourmet(pedidos)
    test_mesas_gourmet(pedidos, 12)
```

```
def test_lee_pedidos(pedidos: list[Pedido])->None:
    print("TEST lee_pedidos")
    print(f"Número de registros leídos: {len(pedidos)}")
    print(f"Los tres primeros registros son:{pedidos[:3]}")
```

Ejercicio 2

```
# Solución por comprensión
def ingreso_total_por_franja(
        pedidos: list[Pedido],
        hora_inicio: int,
        hora_fin: int,
        metodo_pago: str|None= None
        ) -> float:
      return sum(pedido.propina + importe_pedido(pedido)
          for pedido in pedidos
                if pedido.fecha_hora.hour >= hora_inicio and pedido.fecha_hora.hour < hora_fin
                        and (metodo_pago is None or pedido.metodo_pago == metodo_pago))

# Solución con bucles
def ingreso_total_por_franja(
        pedidos: list[Pedido],
        hora_inicio: int,
        hora_fin: int,
        metodo_pago: str|None= None
        ) -> float:
    """
    Devuelve el ingreso total acumulado de los pedidos en una franja horaria determinada.
    """
    ingreso_total = 0.0

    for p in pedidos:
        if hora_inicio <= p.fecha_hora.hour < hora_fin
                and (metodo_pago is None orp.metodo_pago.lower() == metodo_pago.lower()):
            # Sumamos el coste del pedido y la propina
            ingreso_total += coste_total_pedido(p) + p.propina
    return ingreso_total

def coste_total_pedido(pedido: Pedido) -> float:
    """Calcula el coste total de un pedido sumando el coste de cada línea."""
    return sum(linea.cantidad * linea.precio_unitario for linea in pedido.lineas)
```

Test

```
def test_ingreso_total_por_franja(pedidos: list[Pedido], hora_inicio: int, \
                                  hora_fin: int, metodo_pago: str|None=None) -> None:
    print(f"TEST ingreso_total_por_franja '{hora_inicio}-{hora_fin}'"
          f"{' con método de pago ' + método_pago if método_pago else ''}")
    ingreso = ingreso_total_por_franja(pedidos, hora_inicio, hora_fin, método_pago)
    print(f"Ingreso total: {ingreso} €\n")
```

Ejercicio 3

```
# --- Solución 1:
#      Creación de diccionario de agrupación con clave mesa, y valores lista de promedios
#      Creación de diccionario de máximos a partir del anterior

def analisis_propinas_ubicacion(
        pedidos: list[Pedido],
        metodo_pago: str
        ) -> tuple[str, float]:
    """
    Devuelve la ubicación con el promedio de porcentaje de propina más alto
    para un método de pago dado.
    """
    ubicacion_promedios = get_ubicacion_promedios(pedidos, metodo_pago)
    mejor_ubicacion = max(ubicacion_promedios.items(), key=lambda x: x[1])
    return mejor_ubicacion

def get_ubicacion_promedios(pedidos: list[Pedido], metodo_pago: str) -> Dict[str, float]:
    ubicacion_promedios = defaultdict(list)

    for p in pedidos:
        if p.metodo_pago.lower() == metodo_pago.lower():
            coste_productos = coste_total_pedido(p)
            pct_propina = (p.propina / coste_productos) * 100
            ubicacion_promedios[p.mesa].append(pct_propina)

    return  {k, sum(v) / len(v)\
                  for k, v in ubicacion_promedios.items()}
    #También con bucles
    #for k, v in ubicacion_promedios.items():
    #    ubicacion_promedios[k] = sum(v) / len(v)
    #return ubicacion_promedios

# --- Solución 2:
#      Creación de diccionario de agrupación con clave mesa, y valores lista de pedidos
#      Creación de diccionario de máximos a partir del anterior
def analisis_propinas_ubicacion(
     pedidos: list[Pedido],
     metodo_pago: str)
     -> tuple[str, float]:
    d = pedidos_por_mesa(pedidos, metodo_pago)
    d_promedios = {mesa:promedio_porcentajes(lista_pedidos)\
                        for mesa, lista_pedidos in d.items() }
    return max(d_promedios.items(), key=lambda t:t[1])

def pedidos_por_mesa(pedidos:list[Pedido], metodo_pago:str)->dict[int, list[Pedido]]:
    res = defaultdict(list)
    for pedido in pedidos:
        if pedido.metodo_pago == metodo_pago:
            res[pedido.mesa].append(pedido)
    return res
```

```
def promedio_porcentajes(pedidos:list[Pedido])->float:
    return statistics.mean (porcentaje_propina(pedido) for pedido in pedidos )
```

Test

```
def test_analisis_propinas_ubicacion(
      pedidos: list[Pedido],
      metodo_pago: str) -> None:
    print(f"TEST análisis propinas por ubicación para método de pago: {metodo_pago}")
    ubicacion, promedio = analisis_propinas_ubicacion(pedidos, metodo_pago)
    print(f"Ubicación con el promedio de propina más alto: {ubicacion} {promedio:.2f}%)\n")
```

Ejercicio 4

```
# Solución 1:
def producto_mas_asociado(
        pedidos: list[Pedido],
        nombre_producto: str
        ) -> str:
    """
    Identifica qué producto acompaña con mayor frecuencia al producto de referencia.
    """
    contador_asociados = Counter()
    for p in pedidos:
        productos_nombres = {l.producto for l in p.lineas}
        if nombre_producto in productos_nombres:
            # Sumamos al contador todos los demás productos del pedido
            # (excluyendo el de referencia)
            productos_nombres.remove(nombre_producto)
            contador_asociados.update(productos_nombres)
    return contador_asociados.most_common(1)[0][0]

# Solución 2:
def producto_mas_asociado(pedidos: list[Pedido], nombre_producto: str) -> str:
    c = Counter()
    for pedido in pedidos:
        if esta_en_linea(pedido.lineas, nombre_producto):
            for linea in pedido.lineas:
                if not linea.producto == nombre_producto:
                    c[linea.producto] += 1
    return max(c, key=c.get)

def esta_en_linea(lineas: list[LineaPedido], nombre_producto: str) -> bool:
    #existe
    res = False
    for linea in lineas:
        if linea.producto == nombre_producto:
            res=True
            break
    return res
```

Test

```
def test_producto_mas_asociado(pedidos: list[Pedido], nombre_producto: str) -> None:
    print(f"TEST producto_mas_asociado para producto: {nombre_producto}")
    producto = producto_mas_asociado(pedidos, nombre_producto)
    print(f"Producto más asociado: {producto}\n")
```

Ejercicio 5

```
# Solución 1:
def mesas_gourmet(
        pedidos: list[Pedido],
        hora: int|None = None
        ) -> list[str]:
    """
    Devuelve las mesas cuyo gasto medio por pedido supera en un 40%
    al promedio global de la sala.
    """

    # 1. Filtrado inicial por condiciones operativas (exclusión sistemática de la "Barra")
    pedidos_filtrados =
             (p for p in pedidos \
                if p.mesa.lower() != "barra" and (p.fecha_hora.hour == hora or hora is None)) #

    # 2. Cálculo del gasto global de la sala
    gasto_global = gasto_medio_global(pedidos_filtrados)

    # 3. Cálculo de gastos individuales por mesa
    gastos_por_mesa = get_gastos_mesas(pedidos_filtrados)

    # 4. Evaluación del criterio de selección de Julio (> 40% del entorno global)
    mesas_gourmet = list() #

    for mesa, gastos in gastos_por_mesa.items():
        gasto_medio_mesa = sum(gastos) / len(gastos)
        if gasto_medio_mesa > (gasto_global * 1.40):
            mesas_gourmet.append(mesa)

    return  mesas_gourmet


def get_gastos_mesas(pedidos: list[Pedido]) -> Dict[str, list[float]]:
    """
    Devuelve un diccionario con el gasto total por mesa, filtrando por hora si se proporciona.
    """
    gastos_por_mesa = defaultdict(list)

    for p in pedidos:
        gastos_por_mesa[p.mesa].append(coste_total_pedido(p))
    return gastos_por_mesa
```

```python
def gasto_medio_global(pedidos: list[Pedido]) -> float:
    """
    Calcula el gasto medio global de todos los pedidos.
    """
    total_gasto = sum(coste_total_pedido(p) for p in pedidos)
    return total_gasto / len(pedidos) if pedidos else 0.0

# Solución 2:
def mesas_gourmet(pedidos: list[Pedido], hora: int|None = None) -> list[str]:
    d = defaultdict(list)
    num_pedidos_hora = 0
    importe_total = 0
    for pedido in pedidos:
        if pedido.mesa != "Barra" and (hora is None or hora == pedido.fecha_hora.hour):
            i = importe_pedido(pedido)
            d[pedido.mesa].append(i)
            importe_total += i
            num_pedidos_hora += 1

    media_cafeteria = None
    if num_pedidos_hora > 0:
        media_cafeteria = importe_total / num_pedidos_hora
    res=[]
    for mesa, lista_importes in d.items():
        media_mesa = statistics.mean(lista_importes)
        if media_mesa > 1.4 * media_cafeteria:
            res.append(mesa)
    return res
```

Test

```python
def test_mesas_gourmet(pedidos: list[Pedido], hora: int|None = None) -> None:
    print(f"TEST mesas_gourmet para hora: {hora}")
    mesas = mesas_gourmet(pedidos, hora)
    print(f"Mesas gourmet: {mesas}\n")
```